Documento de Arquitectura de Software

Sistema de Gestión de Citas Médicas

Introducción 3

- **Propósito** 3
- **Alcance** 3
- **Público Objetivo** 4

Vista General de la Arquitectura 5

- **Diagrama de Alto Nivel (Diagrama de Contexto, Nivel 1)** 5
- **Principios y Patrones Arquitectónicos** 6

Vistas Arquitectónicas (Modelo 4+1) 7

- **Vista Lógica** 7
- **Vista de Procesos** 11
- **Vista de Desarrollo** 12
- **Vista de Despliegue** 14
- **Casos de Uso** 15

Funcionalidades por Aplicación 18

- **Aplicación Web (Pacientes)** 18
- **Aplicación de Escritorio (Médicos y Administrativos)** 18

Diagrama de Base de Datos (ER) 19

Componentes Claves 20

Decisiones Arquitectónicas 21

Requisitos no Funcionales 22

Gabriel Alcequiez (2025-1062)

Riesgos Técnicos y Mitigaciones **23**

Conclusión **23**

Introducción

Propósito

El propósito de este documento es definir y dar a conocer la Arquitectura de Software, que se utilizará en el Sistema de Gestión de Citas Médicas, ofreciendo una vista general de la misma, una vista con el modelo 4+1, dando a conocer los componentes y tecnologías claves de la arquitectura.

Alcance

El alcance de este documento comprende la descripción detallada de la arquitectura técnica del Sistema de Gestión de Citas Médicas (SGCM). Se documentan las decisiones de diseño para el backend en .NET (API REST bajo **Clean Architecture**), la **estrategia de persistencia y modelado de datos**, y sus aplicaciones cliente actuales: **Web (para Pacientes)** y **Escritorio (para Médicos y Personal Administrativo)**.

Asimismo, se especifica la descomposición del sistema siguiendo el **Modelo de Vistas 4+1**, detallando la integración actual con servicios de notificaciones y los **mecanismos de extensibilidad** que preparan al sistema para futuras expansiones sin afectar el núcleo del negocio, incluyendo:

- La incorporación de pasarelas de pago.
- La habilitación de acceso Web para Médicos, permitiendo la gestión remota.
- El desarrollo de una futura aplicación móvil nativa para pacientes.
-

Público Objetivo

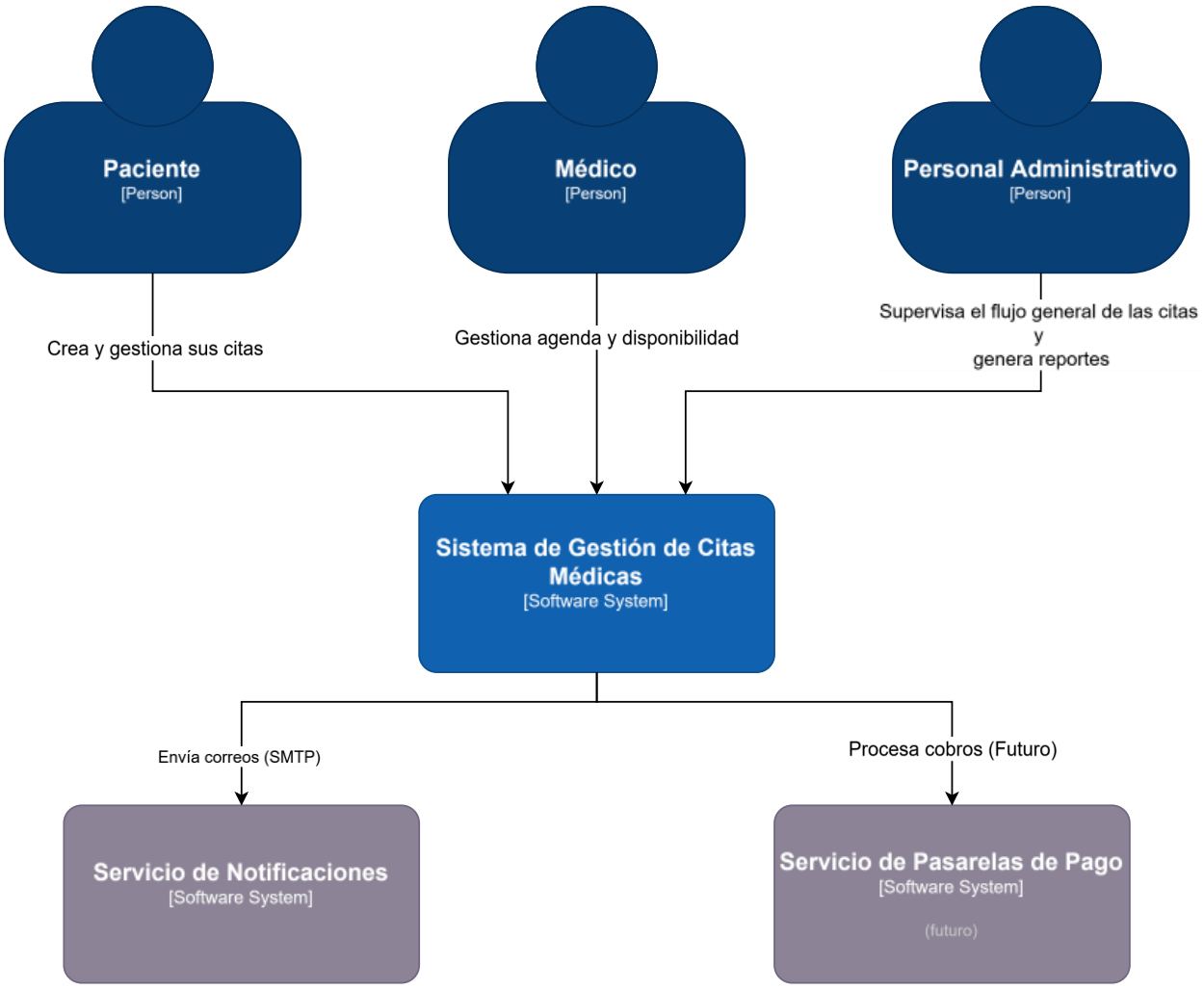
Este SAD está dirigido a los roles técnicos involucrados en el ciclo de vida del SGCM, encargados de hacer el sistema realidad:

- Arquitectos de Software
- Desarrolladores de Software (Backend, Web y Desktop)
- Evaluadores del Proyecto y Stakeholders

Vista General de la Arquitectura

La arquitectura presenta una visión de la solución tecnológica propuesta para el Sistema de Gestión de Citas Médicas (SGCM). La arquitectura ha sido diseñada siguiendo estrictamente el patrón de la Arquitectura Limpia (Clean Architecture) con un enfoque centrado en el dominio y distribuido, priorizando la integridad de los datos y la escalabilidad.

Diagrama de Alto Nivel (Diagrama de Contexto, Nivel 1)



En este diagrama, el SGCM actúa como el núcleo central que orquesta los flujos de trabajo para tres actores clave: los **Pacientes** (solicitud y gestión de citas), los **Médicos** (control de agenda y disponibilidad) y el **Personal Administrativo** (supervisión del flujo operativo).
Adicionalmente, se modelan las dependencias externas del sistema, destacando la integración con el **Servicio de Notificaciones** para el envío de correos transaccionales (SMTP) y la previsión arquitectónica para una futura conexión con **Pasarelas de Pago**, asegurando que el diseño actual soporte dicha expansión sin refactorizaciones mayores.

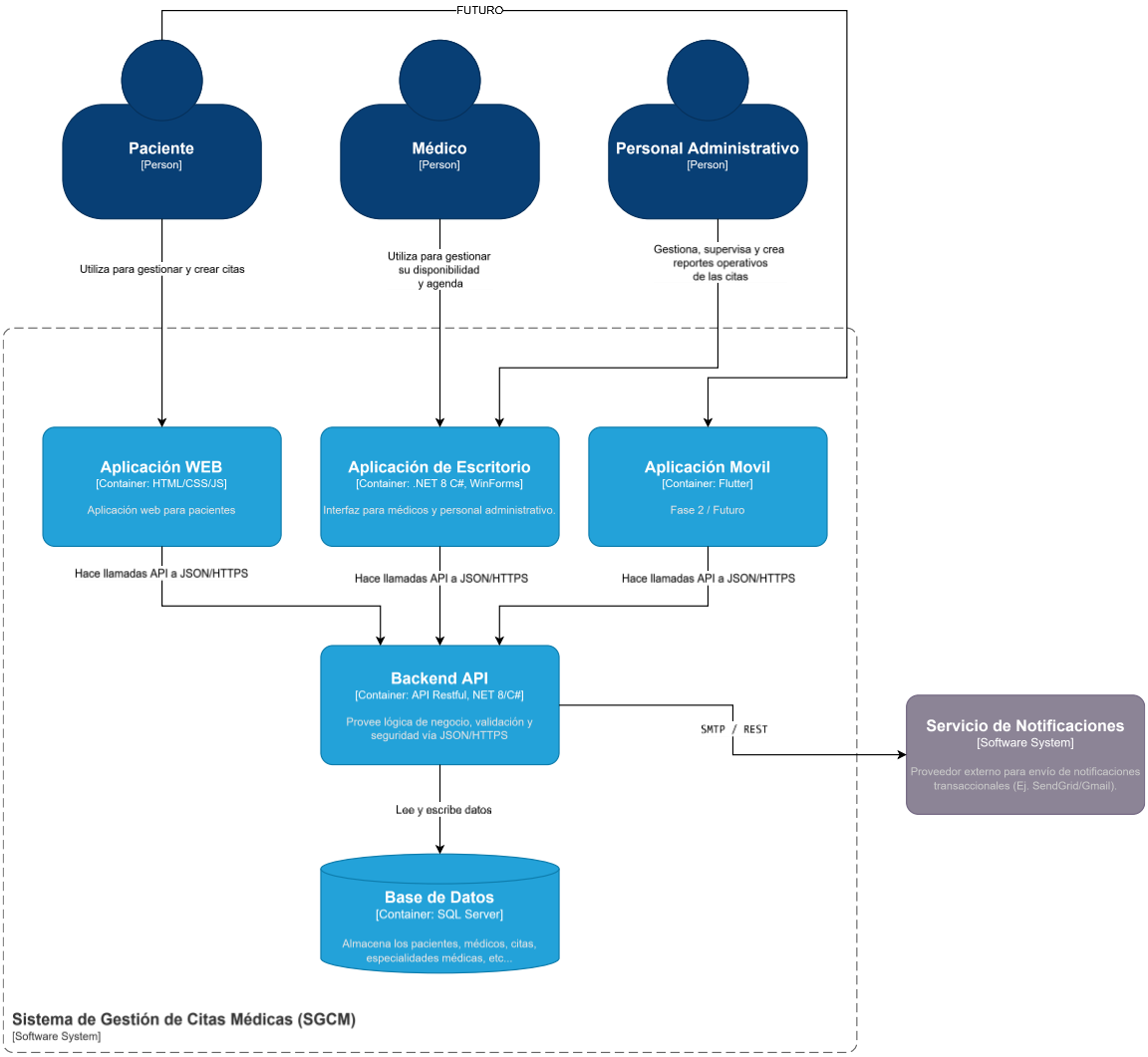
Principios y Patrones Arquitectónicos

La arquitectura del sistema se rige fundamentalmente por la **Separación de Responsabilidades** (SoC) y los principios **SOLID**, estableciendo una estricta división entre la interfaz de usuario, la lógica de negocio y el acceso a datos. Se aplica rigurosamente la **Inversión de Dependencias** (DIP) para garantizar que el núcleo del dominio permanezca agnóstico a la infraestructura externa (como la base de datos).

Vistas Arquitectónicas (Modelo 4+1)

Vista Lógica

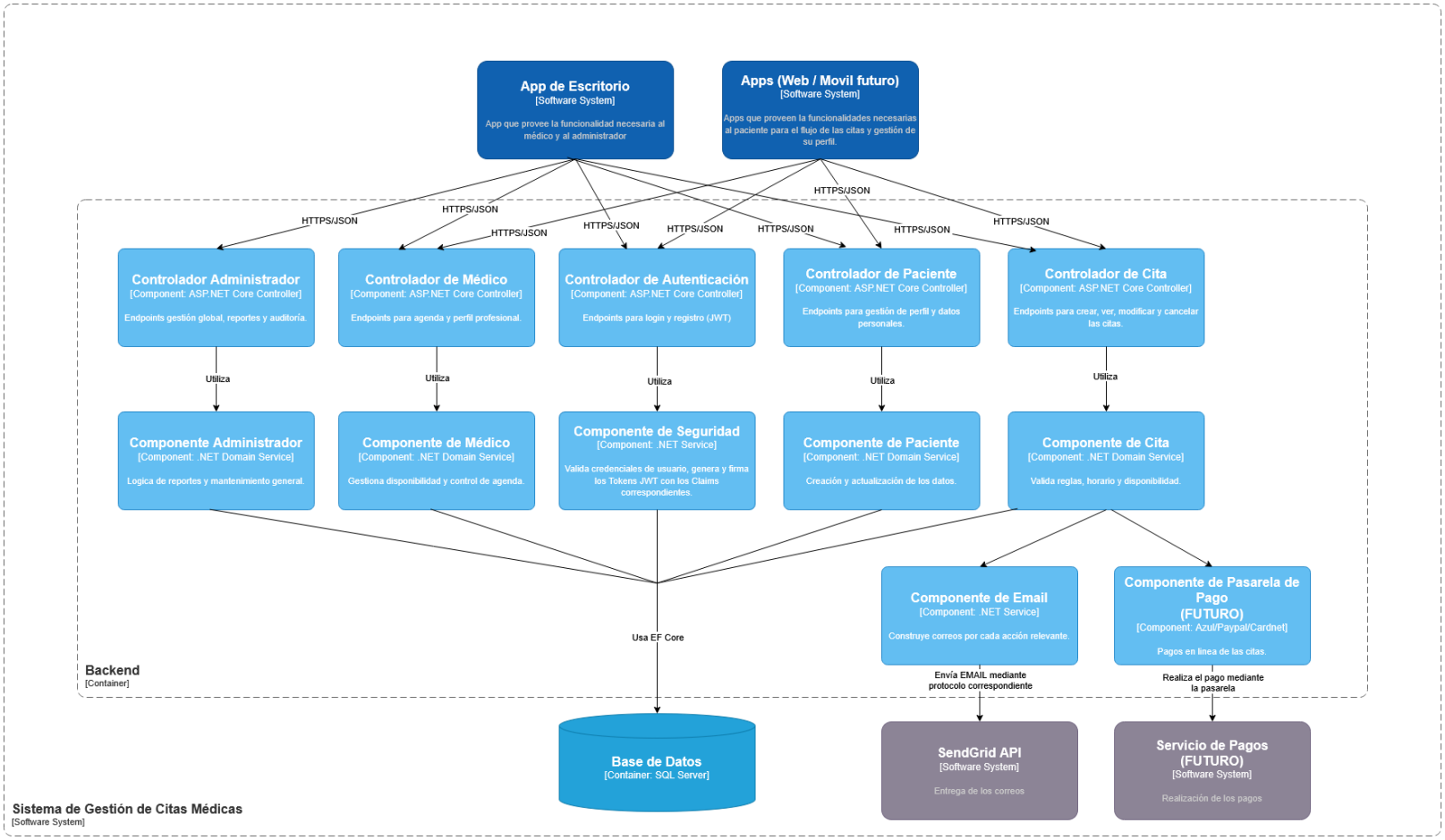
- **Diagrama de Contenedores (nivel 2)**



Desglosando la solución en unidades desplegables independientes y sus tecnologías específicas. Se adopta un enfoque cliente-servidor donde la lógica de presentación se distribuye en dos clientes especializados: una **Aplicación Web** para la accesibilidad de los pacientes y una **Aplicación de Escritorio** para la gestión administrativa intensiva. Ambos contenedores

interactúan mediante peticiones HTTPS/JSON con un único **API Backend** desarrollado en **.NET 8**, el cual actúa como el núcleo central de la lógica de negocio y seguridad. Finalmente, este backend gestiona exclusivamente la persistencia de datos conectándose a un contenedor de base de datos **SQL Server**, garantizando la centralización e integridad de la información clínica.

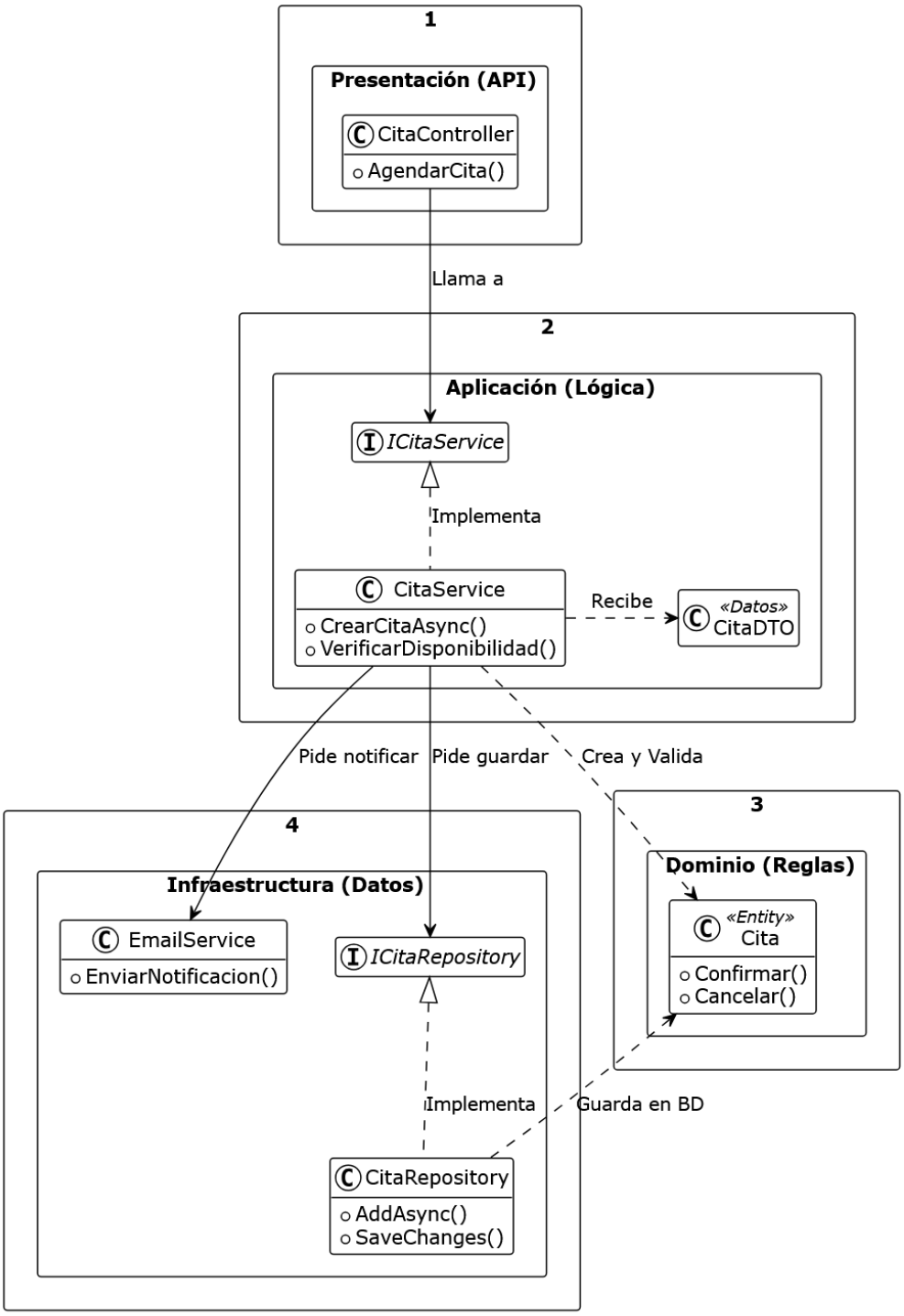
- **Diagrama de Componentes (nivel 3)**



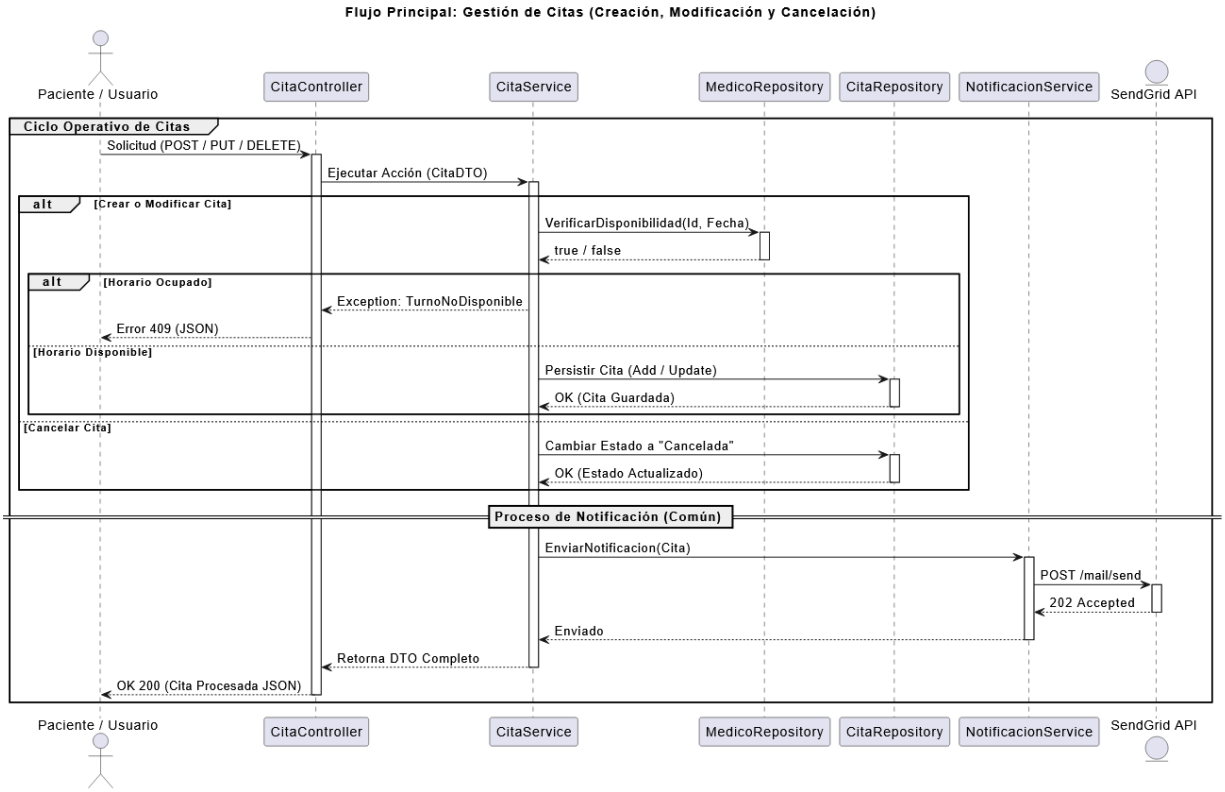
El diagrama de componentes ilustra la descomposición interna del contenedor **Backend API** bajo la plataforma .NET 8, siguiendo el principio de Separación de Responsabilidades. La arquitectura distingue claramente entre la **Capa de Entrada** (Controladores ASP.NET Core), encargada de exponer los endpoints HTTPS/JSON para las aplicaciones cliente (Web, Escritorio y futura Móvil), y la **Capa de Lógica** (Componentes de Dominio), que encapsula las reglas de negocio, validaciones y seguridad. Estos componentes actúan como el núcleo del sistema, orquestando tanto la persistencia de datos en **SQL Server** como la integración con servicios externos (SendGrid y futuras pasarelas de pago), garantizando un bajo acoplamiento y facilitando la mantenibilidad del código.

Gabriel Alcequiez (2025-1062)

- **Diagrama de Código (nivel 4)**

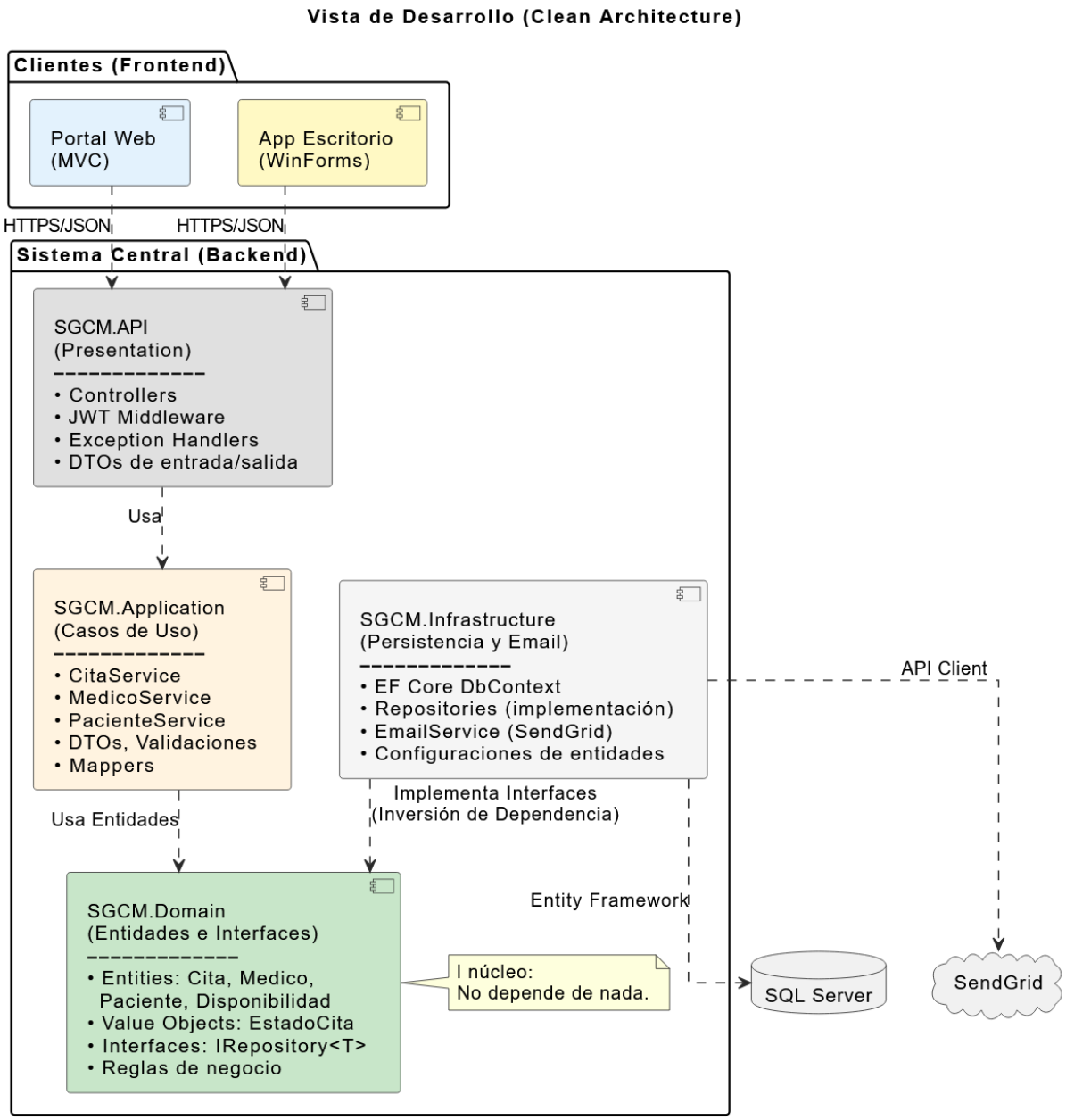


Vista de Procesos



Este diagrama detalla el ciclo operativo completo de las citas. Se observa cómo el CitaService orquesta la validación de disponibilidad para creaciones y cambios, así como la actualización de estados para cancelaciones, garantizando que toda operación finalice con una notificación al actor correspondiente y un registro íntegro en la base de datos.

Vista de Desarrollo



- **Estructura del código**

Solution 'SGCM'

/SGCM.API -> Puntos de entrada (Controllers)

/SGCM.Domain (Core) -> Entidades del núcleo (Core), Interfaces de repositorios, Clases Base.

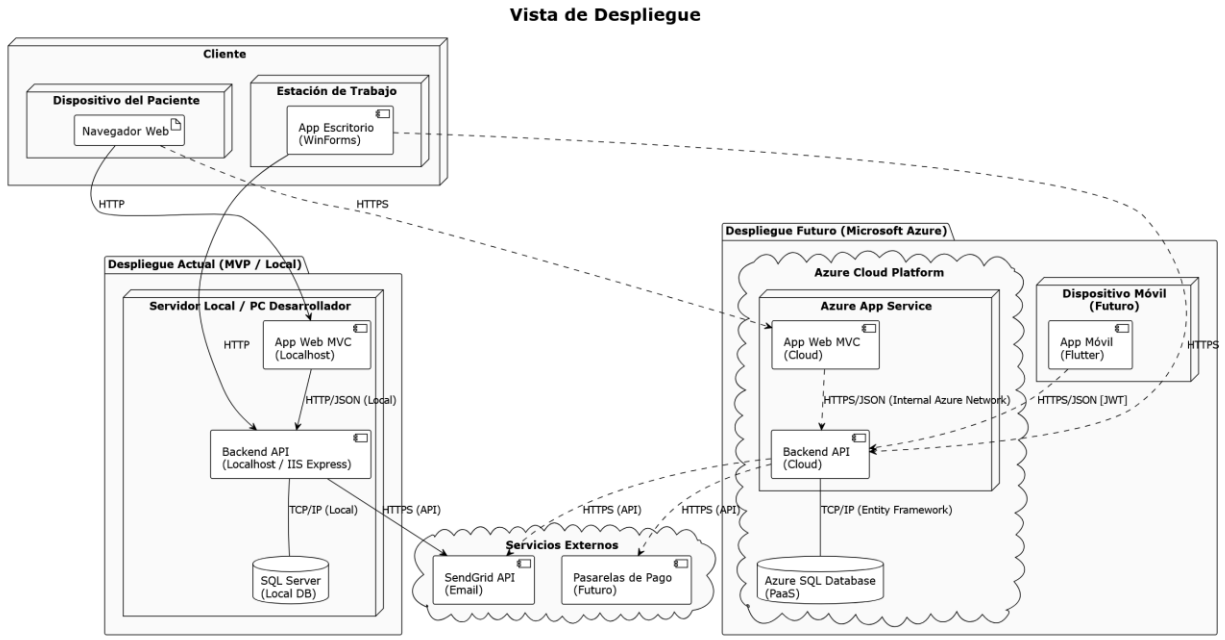
/SGCM.Application -> Casos de uso (Services), DTOs, Validaciones, Mappers.

/SGCM.Infrastructure -> Implementación de persistencia (EF Core), Adaptadores externos (Email)

- **Tecnologías**

Categoría	Tecnología	Descripción
Lenguaje de Programación	C#	Lenguaje base para toda la lógica de negocio, controladores y persistencia.
API	ASP.NET Core Web API (.NET 8)	Implementación de servicios RESTful que exponen la lógica de negocio a los clientes.
Cliente Web	ASP.NET Core MVC / HTML5 & CSS3	Interfaz de usuario web que consume la API para la gestión de citas y pacientes.
Base de Datos	Microsoft SQL Server	Motor de base de datos relacional empresarial, altamente integrado con el ecosistema .NET.
Acceso o persistencia de datos	Entity Framework Core	Manejador de persistencia (Code-First) optimizado para trabajar nativamente con SQL Server.
Servicios Externos	SendGrid API	Servicio en la nube para el envío transaccional de notificaciones por correo electrónico.
Servicios de Pago **(FUTURO)**	Azul, Paypal, Cardnet	Servicio para permitir los pagos desde el sistema.

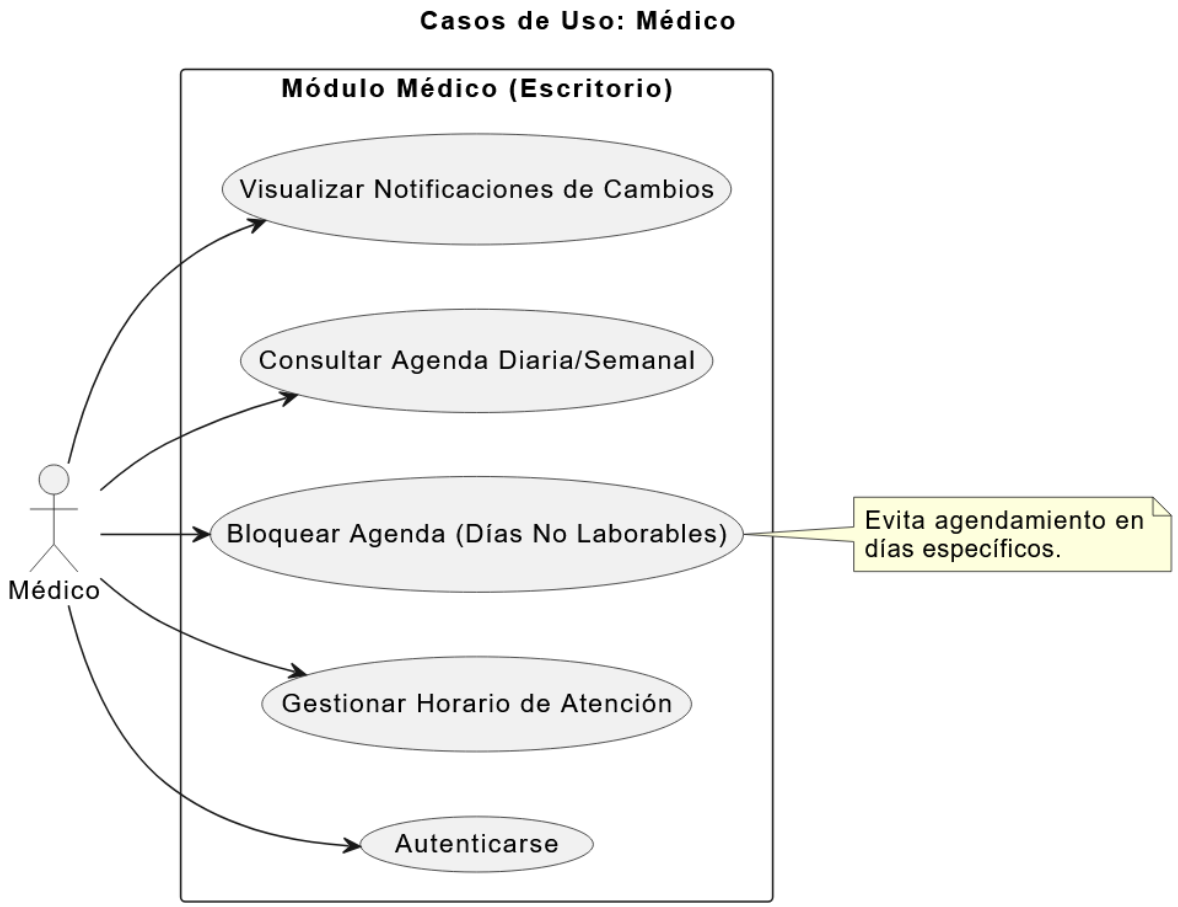
Vista de Despliegue



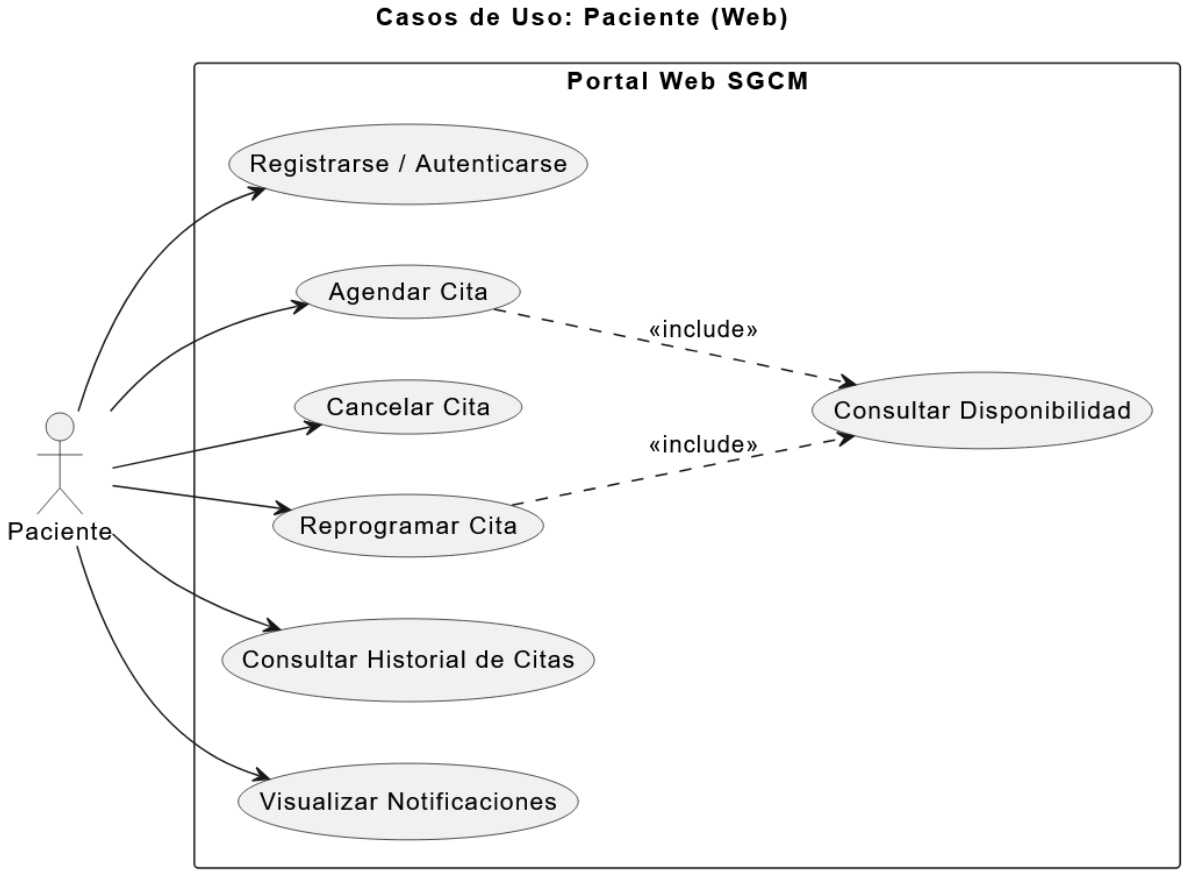
Se plantea un modelo de despliegue en dos fases: una **Fase Actual (On-Premise)** para el lanzamiento inicial y una **Fase Proyectada o Futuro (Cloud)** que migra la infraestructura a servicios gestionados de Microsoft Azure para garantizar alta disponibilidad.

Casos de Uso

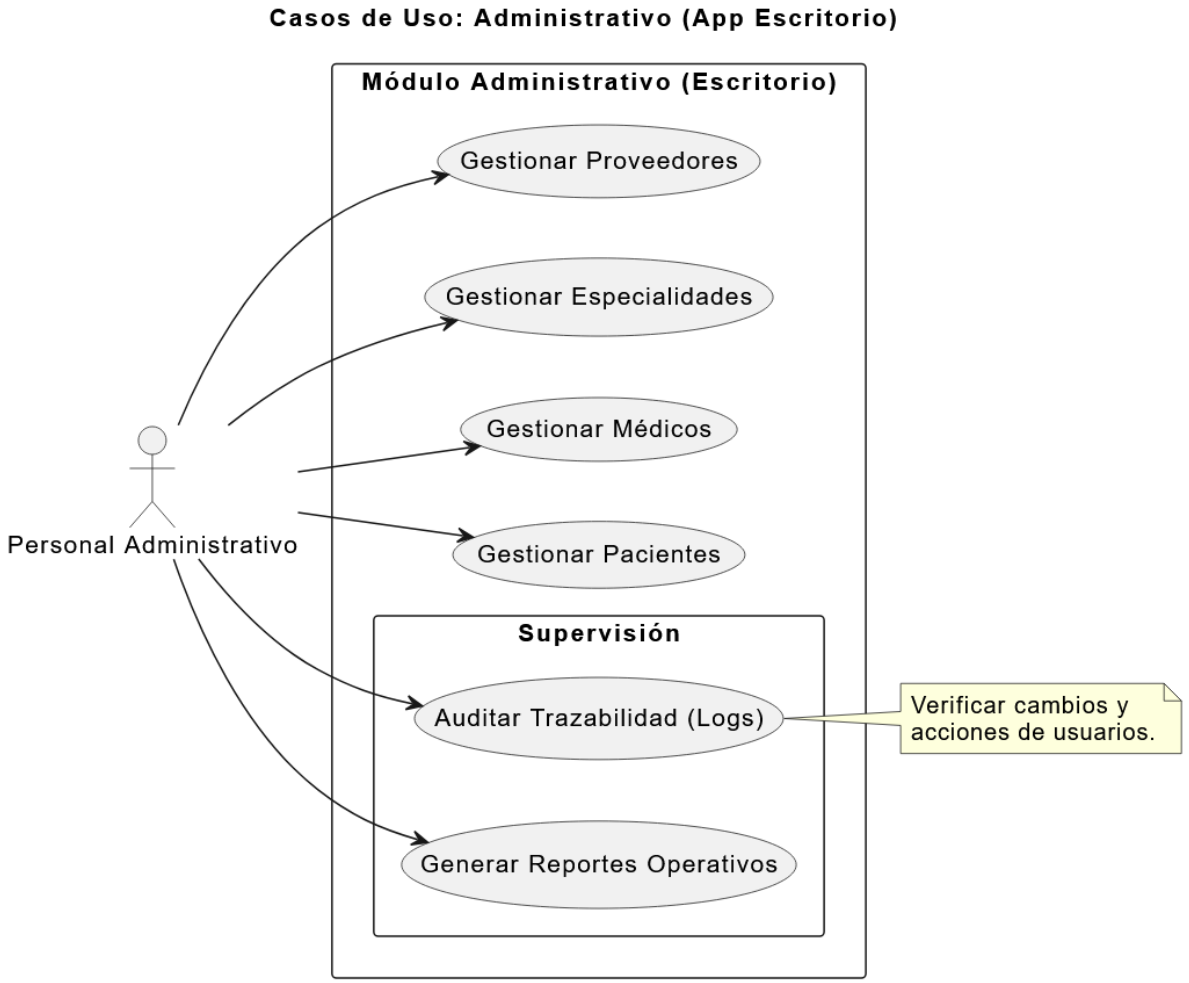
- **Actor Médico**



- **Actor Paciente**



- **Actor Personal Administrativo**



Funcionalidades por Aplicación

Aplicación Web (Pacientes)

Registro y autenticación de usuarios

Consulta de disponibilidad de médicos

Agendamiento de citas

Cancelación y reprogramación de citas

Consulta de historial de citas

Aplicación de Escritorio (Médicos y Administrativos)

Autenticación de usuarios y gestión de roles

Gestión de agenda y horarios (Médico)

Bloqueo de días no laborables (Médico)

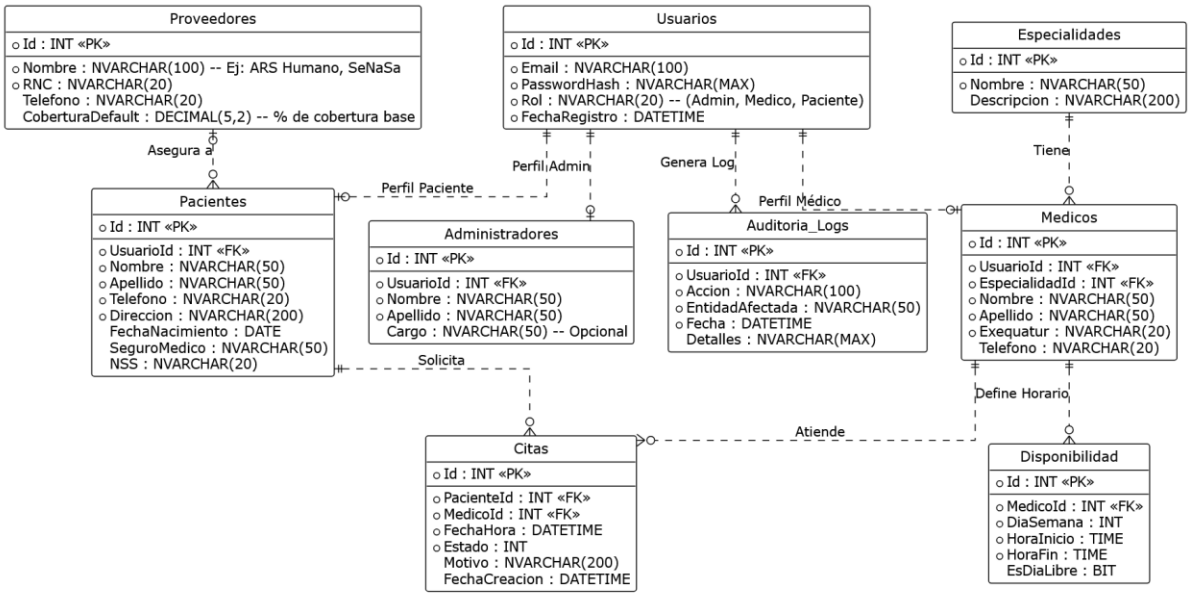
Gestión de médicos y especialidades (Admin)

Gestión de pacientes (Admin)

Auditoría y consulta de logs (Admin)

Generación de reportes operativos (Admin)

Diagrama de Base de Datos (ER)



Gabriel Alcequiez (2025-1062)

Componentes Claves

Componente	Descripción	Justificación
Aplicación Web	Interfaz en navegador para que los pacientes gestionen sus citas.	Acceso universal y facilidad de uso para el usuario final.
Aplicación de Escritorio	Software robusto para médicos y personal administrativo (WinForms).	Mayor eficiencia y control en tareas administrativas intensivas.
API Backend (.NET 8)	Núcleo central que expone la lógica de negocio vía RESTful.	Centraliza la seguridad y las reglas de negocio para todos los clientes.
Base de Datos (SQL Server)	Motor relacional que almacena citas, médicos y pacientes.	Garantiza la integridad, consistencia y trazabilidad de la información.
Servicio de Notificaciones	Integración externa con SendGrid para envío de correos.	Automatiza avisos y recordatorios sin sobrecargar el servidor principal.
Pasarelas de Pago (Futuro)	Integración proyectada con servicios como PayPal o Azul.	Permite la escalabilidad del modelo de negocio a pagos en línea.

Estos son los componentes clave con los que debe contar el sistema, para que en su implementación se logre conseguir el MVP (Producto Mínimo Viable) .

Decisiones Arquitectónicas

Decisión	Alternativas a la decisión	Justificación	Fecha
Arquitectura Limpia	N-Capas Simple	Facilita la mantenibilidad y permite que el núcleo del negocio sea independiente de la infraestructura y la UI.	30/01/2026
API Backend en .NET 8 (WEB Api)	gRPC, GraphQL	Aprovecha el ecosistema robusto de C# y la alta compatibilidad con SQL Server y Azure (para despliegue en el futuro).	30/01/2026
Base de Datos Relacional (SQL Server)	MySQL	Garantiza integridad transaccional (ACID) crítica para evitar duplicidad en citas médicas y aprovecha ecosistema de Microsoft con c#.	30/01/2026
Notificaciones vía SendGrid	Servidor SMTP propio.	Asegura la entregabilidad de los correos de confirmación y recordatorios sin bloquear el flujo principal del sistema.	30/01/2026
Autenticación con JWT	Cookies de sesión, OAuth2 externo.	Permite una gestión de seguridad estandarizada y persistente tanto para la App Web como para la de Escritorio.	30/01/2026

Estas decisiones garantizan la viabilidad técnica y operativa del sistema mediante:

- **Sincronización Total:** Centraliza la lógica para que todos los clientes (Web y Escritorio) consuman datos idénticos en tiempo real.
- **Independencia Técnica:** Permite actualizar tecnologías o proveedores externos sin afectar el núcleo del negocio.
- **Seguridad y Auditoría:** Asegura un control de acceso robusto (JWT) y una persistencia de datos íntegra y auditable.
- **Alta Disponibilidad:** Facilita el crecimiento del sistema y asegura que esté operativo cuando los pacientes lo necesiten.

Gabriel Alcequiez (2025-1062)

Requisitos no Funcionales

Atributo	Especificación
Seguridad	Implementación de JWT (JSON Web Tokens) para autenticación.
Disponibilidad	El sistema debe garantizar un tiempo de actividad del 99%, asegurando la continuidad operativa del agendamiento y la consulta de citas.
Rendimiento	Procesamiento de agendamientos en menos de 3 segundos mediante el uso de índices en SQL Server y servicios optimizados en .NET 8.
Usabilidad	Interfaz intuitiva y coherente diseñada para navegación web (HTML5/CSS3) y escritorio (WinForms).
Trazabilidad	Registro inalterable de cada cambio en la agenda médica mediante Logs de actividad y auditoría persistente.
Integridad	Uso de Transacciones ACID en SQL Server para asegurar que la agenda médica sea siempre consistente y sin duplicidad.
Escalabilidad	Capacidad del sistema para atender a más usuarios y manejar más citas a medida que la clínica crezca sin perder velocidad.

Riesgos Técnicos y Mitigaciones

Riesgo	Impacto	Mitigación
Condición de carreras (ósea, citas duplicadas en el mismo horario).	ALTO	Implementar restricciones de unicidad (Unique Index) en la base de datos y validaciones de concurrencia en la capa de servicios.
Perdida de Conectividad (Clinica sin internet para la app desktop)	Medio	Manejo de excepciones y estados "fuera de línea" con mensajes claros al usuario para evitar bloqueos del sistema.
Fallo en servicio externo (Caída de SendGrid para notificaciones).	Bajo	Implementar una cola de mensajería o proceso asíncrono; si el correo falla, la cita se registra igual y se reintenta el envío luego.
Latencia en consultas (Demora al cargar agendas con muchos datos).	Medio	Uso de índices en SQL Server y paginación de resultados en la API para cumplir con tiempos de respuesta menores a 3 segundos.

Conclusión

La arquitectura definida en este documento establece una base sólida, escalable y segura para el Sistema de Gestión de Citas Médicas. Mediante la implementación de Clean Architecture y una API centralizada en .NET 8, se garantiza la independencia de la lógica de negocio y la sincronización total entre las plataformas web y de escritorio.

Este diseño no solo mitiga los riesgos críticos, como la duplicidad de citas y la falta de trazabilidad, sino que también asegura que el sistema sea fácil de mantener y evolucionar ante futuros requerimientos de la clínica.